

MAT 167: Final Project

Google's PageRank

Group 05

By: Shihao Huang, Alex Ren, Alexander Cai

hshuang@ucdavis.edu, asren@ucdavis.edu, alecai@ucdavis.edu

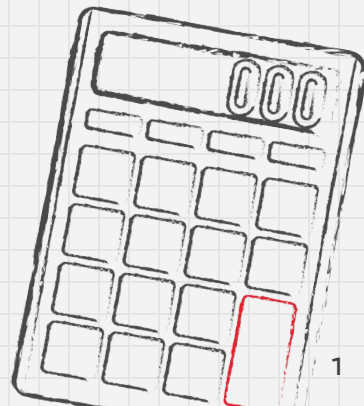
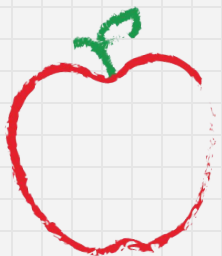


Table of contents

01

Introduction

Overview of Topic

02

Foundation

What is the PageRank
Algorithm?

03

The Math

How does the Algorithm
Mathematically work?

04

The Algorithm

Implementing PageRank

05

Conclusion

Our results

1. Introduction

Importance:

Created by Sergey Brin & Larry Page in 1998[1]:

- One of the first Search Engine Optimization Algorithms to emphasize backlinks (linking a site to another). [2]
 - **Idea:** Anytime you search for information or are recommended anything, it's most likely built using a PageRank Algorithm[3].
 - Therefore, most internet applications are built using aspects of PageRank.

Without the PageRank Algorithm, the Internet would be an informational wild west

2. What is the PageRank Algorithm?

- Helps Google determine importance of a webpage based on number of “votes”[1].
 - Votes vary in weight (The more reputable, the more weight it holds).
 - **Exp:** A post by the New York Times is more reputable than a small blog. [2]
- Each page distributes its influence across outlinks.
 - The more outlinks a page has, the less weight each vote holds.
 - **Exp:** NY Times have lots of outlinks, therefore each outlinks weight is lessened.

Rank pages based on the quality and quantity of links.

03

The Math

Formulas and Breakdown of concepts.

Note: To avoid repetition, most images are pulled directly from [1], Descriptions of concepts have also been influenced by [1].

3.1: The PageRank Algorithm

$$r(P) = \sum_{Q \in \mathcal{B}_P} \frac{r(Q)}{|Q|},$$

- **r(P):** Rank of a page.
- **r(Q):** Weight of a page.
- **|Q|:** Total number of outlinks in a page.
- **B_p:** Set of inlinks to page P. [List of pages]

Rank = Weight/Total links on page.

3.1: The PageRank Algorithm

- Update Rule:
 - Suppose we have n pages: $P_1, \dots, P_n$
 - We initialize each page P_i , where $i = 1, \dots, n$, with the same rank ($1/n$) to ensure that they start off uniformly
 - The algorithm proceeds to iteratively update the rank of P_i for each step j , which depends on the rank calculated from the previous step, $j-1$, divided by the number of outlinks from page Q
 - The number of steps j depends on when the rank of P_i converges

$$r_j(P_i) = \sum_{Q \in \mathcal{B}_{P_i}} \frac{r_{j-1}(Q)}{|Q|}, \quad \text{for } j = 1, 2, \dots$$

3.1: The PageRank Algorithm

- Update Rule:

- In vector form, we have π_{j-1}^T , which contains the ranks of each page, and we update iteratively until convergence by multiplying the previous $j-1$ th vector by the “Google” matrix G and then normalize using the l_1 norm, ensuring that the probabilities sums to 1)

$$\pi_j^T = \pi_{j-1}^T G, \quad \pi_j = \pi_j / \|\pi_j\|_1$$

- This method is called Power Iteration, which we cover in more detail later
- The i th entry in the vector represents the “PageRank” of page P_i
- In the next slide, we will explain the “Google” matrix G

3.2: Raw Google Matrix

$$g_{ij} = \begin{cases} 1/|P_i| & \text{if } P_i \text{ links to } P_j; \\ 0 & \text{otherwise.} \end{cases}$$

- **G**: the Raw Google Matrix
- The entries of g_{ij} are essentially “binary” (0/1), while also scaled by probabilities if 1
 - **$1/|P_i|$** : The probability of user on page j following an outlink to page i .
 - **0**: No outlink exists from page j to i .

3.2: Raw Google Matrix

Legend:

Column: Current Page (j)

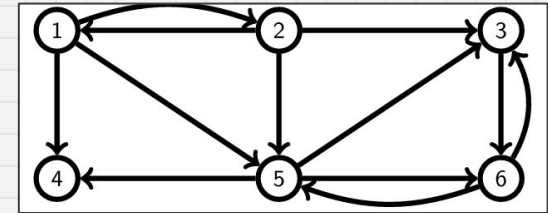
Row: Page being linked (i)

$$G = \begin{bmatrix} 0 & 1/3 & 0 & 1/3 & 1/3 & 0 \\ 1/3 & 0 & 1/3 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1/3 & 1/3 & 0 & 1/3 \\ 0 & 0 & 1/2 & 0 & 1/2 & 0 \end{bmatrix}$$

Raw Google Matrix Exp. [1]

Example:

- **Given the following matrix/graph:**
- **First Page:** links to 2nd, 4th and 5th page (33% to outlink either)
 - Inlinks from page 2.
- **Third Page:** links to 6th page (100% to outlink)
 - Inlinks from 2nd, 5th, and 6th pages.



Web Graph of the Matrix. [1]

Outlink: If I am on page j, what pages can I go to?

Inlink: If I am on page j, what pages point me to j?

- Note: Rows with all zeros indicate that pages have no outlinks.

3.3: Power Iteration (Vector)

$$\pi_j^T = \pi_{j-1}^T G, \quad \pi_j = \pi_j / \|\pi_j\|_1$$

Rules:

- Every
Element ≥ 0

- Sum of all
elements = 1.

After G matrix is found, apply Power Method:

- 1) Initialize v_0 as a uniform distribution of all pages.
 - $v_0 = [1/n, 1/n, \dots]$
- 2) Multiply the probability the vector into the Row G Matrix:
 - $v_1 = v_0 * G$
- 3) Repeat process until either $v_{k+1} \approx v_k$, or maximum iterations are met. [4]

Quantifies how valuable a page is based on the number and significance of inlinks, while inversely scaled by number of outlinks.

3.4: Dangling Nodes

$$G = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Dangling Node Exp. [5]

Pages with no outlinks are called Dangling Nodes

- **Problems:**

- Destroys row stochasticity (sum of all rows $[i] \neq 1$)
- Drains PageRank, and results all pages ≈ 0 .
 - Happens due to repeated iterations of the power method.

- **Solution:**

- 1) Replace the 0 row with uniform probability vector($1/n$)
 - Models how a user continues to use the internet, picking a new $[n]$ page.
 - Updates/corrects existing G Matrix.


$$G = \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} \\ 1 & 0 & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}$$

G Matrix after
applying solution.

If a page has no outlinks -> send the user to a random page.

3.5: Irreducibility

Irreducible: Every Webpage can be accessed by another, no matter where you start.

- Is only observable and applied after handling Dangling Nodes.

Mathematically: A path exists for node i to node j (for all i,j starting points in GMatrix)

- Prevents a user from being trapped on one section of the internet

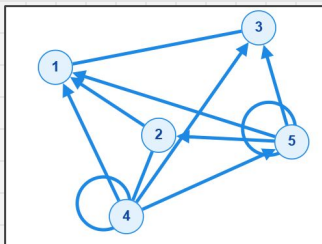
These are two
separate
examples*

Reducible

Despite our matrix having no dangling nodes, it is reducible:

$$G = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ \frac{1}{2} & 0 & 0 & \frac{1}{2} & 0 \\ 1 & 0 & 0 & 0 & 0 \\ \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} & \frac{1}{5} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & 0 & \frac{1}{4} \end{bmatrix}$$

Reducible Matrix Exp.[5]



Reducible Graph Exp. [6]

Note: 1 <-> 3

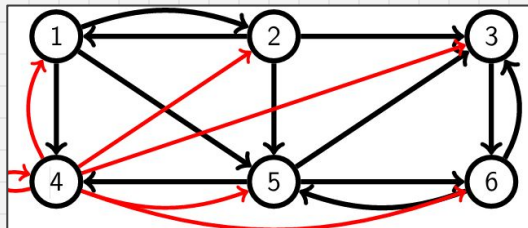
The user is stuck.

Irreducible

$$G = \begin{bmatrix} 0 & 1/3 & 0 & 1/3 & 1/3 & 0 \\ 1/3 & 0 & 1/3 & 0 & 1/3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1/6 & 1/6 & 1/6 & 1/6 & 1/6 & 1/6 \\ 0 & 0 & 1/3 & 1/3 & 0 & 1/3 \\ 0 & 0 & 1/2 & 0 & 1/2 & 0 \end{bmatrix}$$

Irreducible Matrix Exp. [1]

After addressing dangling node (4), our matrix not only is corrected, but irreducible! (No dead-ends)



Irreducible Graph Exp. [1]

3.5: Irreducibility

If matrix is reducible, apply “teleportation”.

$$\tilde{G} = \alpha G + (1 - \alpha)E \quad \text{where } 0 < \alpha < 1 \text{ and } E := \frac{1}{n} \mathbf{1}_n \mathbf{1}_n^T,$$

Idea: Randomly send the user to any page.

Breaking the formula down:

- **α :** Probability of a user clicking on an outlink
- **G :** The Corrected Matrix (after applying solution for dangling nodes).
- **$(1-\alpha)$:** The probability of user not clicking an outlink, navigating to a random page.
- **E :** The uniform Teleportation Matrix (All links are equally likely for every i)
 - Ensures $G_{\sim}$ becomes both stochastic and irreducible [7]
- **$G_{\sim}$:** Stochastic/irreducible form of G .

$$E = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \end{bmatrix}$$

Teleportation
Matrix Exp. [5]

Applying Irreducibility also ensures the Power Method converges to a stationary PageRank Vector (π^T), preventing an endless cycle. [7]

Putting it all together:

Formula	Purpose	Relevance:
$r_j(P_i) = \sum_{Q \in \mathcal{B}_{P_i}} \frac{r_{j-1}(Q)}{ Q }, \quad \text{for } j = 1, 2, \dots$	Finds the rank of a page. Processes pages iteratively. (Gives you the final ranking)	Filters pages, sorting and only displaying the most reputable.
$g_{ij} = \begin{cases} 1/ P_i & \text{if } P_i \text{ links to } P_j; \\ 0 & \text{otherwise.} \end{cases}$	All possibilities of i pages to j outlinks.	Tells you which pages are linked to another.
$\boldsymbol{\pi}_j^\top = \boldsymbol{\pi}_{j-1}^\top G, \quad \boldsymbol{\pi}_j = \boldsymbol{\pi}_j / \ \boldsymbol{\pi}_j\ _1$	Stores current estimated ranking for each page (computes rankings)	Describes how valuable each page is in your Google Matrix.
$\tilde{G} = \alpha G + (1 - \alpha)E \quad \text{where } 0 < \alpha < 1 \text{ and } E := \frac{1}{n} \mathbf{1}_n \mathbf{1}_n^\top$	Solves and fixes reducibility.	Ensures all pages can be accessed by one another. (User can access all pages given any starting point)

04

The Algorithm

Implementation of PageRank

Our Keywords/Google Network:

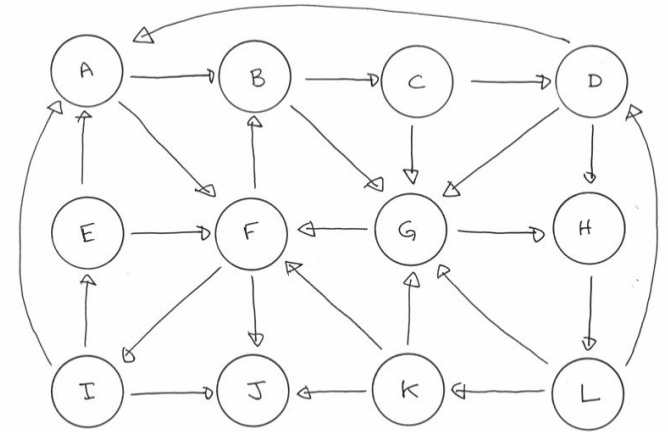
FINAL PROJECT WEBPAGE KEYWORDS #3: TREES

MAT 167 001 / University of California, Davis / Spring Quarter 2026

25 May 2026

Here is a list of the keywords for each webpage.

WebPage A: Trees For All	Ash, Butternut, Cherry, Elm, Katsura, Magnolia, Teak, Ginkgo, Eucalyptus
WebPage B: The Weeping Willow	Butternut, Fir, Hickory, Magnolia, Pine, Willow, Redwood, Sassafras
WebPage C: Leaf Fall	Ash, Elm, Hickory, Katsura, Oak, Ginkgo, Redwood, Eucalyptus
WebPage D: Shady Place	Butternut, Cherry, Fir, Spruce, Teak, Aspen, Sassafras
WebPage E: The Prickly Pine	Cherry, Hickory, Oak, Pine, Willow, Redwood, Eucalyptus
WebPage F: Good Wood	Ash, Fir, Magnolia, Spruce, Ginkgo, Redwood, Aspen, Sassafras
WebPage G: Lumber Jack Man	Ash, Butternut, Oak, Spruce, Ginkgo, Redwood
WebPage H: Timber!	Ash, Cherry, Hickory, Willow, Redwood, Aspen, Eucalyptus
WebPage I: Fine Grains	Elm, Fir, Katsura, Magnolia, Pine, Spruce, Sassafras, Eucalyptus
WebPage J: The Warped Board	Magnolia, Oak, Willow, Redwood, Aspen, Sassafras, Eucalyptus
WebPage K: Evergreen Plants	Cherry, Elm, Fir, Hickory, Teak, Ginkgo, Redwood, Sassafras
WebPage L: Sappy Planks	Butternut, Elm, Katsura, Oak, Pine, Spruce, Teak, Ginkgo, Aspen, Sassafras



Google Network #10

Assigned Keywords by Professor Varn [8]

Assigned Google Network by Professor Varn [9]

4.1 Input data + Constructing Matrix G

Page	Title	Outlinks
A	Trees For All	B, F
B	The Weeping Willow	C, G
C	Leaf Fall	D, G
D	Shady Place	A, G, H
E	The Prickly Pine	A, F
F	Good Wood	B, I, J
G	Lumber Jack Man	F, H
H	Timber!	L
I	Fine Grains	A, E, J
J	The Warped Board	None
K	Evergreen Plants	F, G, J
L	Sappy Planks	D, G, K

Input Data

Page	Outlinks	Raw row sum
A	2	1
B	2	1
C	2	1
D	3	1
E	2	1
F	3	1
G	2	1
H	1	1
I	3	1
J	0	0
K	3	1
L	3	1

Raw Google Matrix (G)

Raw eigenvalue check:
Dominant eigenvalue before fixing dangling page is 0.9113, which is not 1, so the raw matrix is not enough.

Row-sum check:
All rows sum to 1 except Page J, which sums to 0.

This matches the project requirement: first check whether raw rows add to 1 or 0.

4.2 Fixing the dangling webpage

Before fixing:

J row = $[0, 0, \dots, 0]$



After fixing:

J row = $[1/12, \dots, 1/12]$

Result:

After fixing, every row of G
sums to 1. This makes G row
stochastic.

4.3 Power Method Check

We also used the power method instead of only using eigendecomposition.

Starting vector:

$$\pi_0 = [1/12, 1/12, \dots, 1/12]$$

Stopping rule:

Stop when $\|\pi_j - \pi_{j-1}\| \leq 10^{-8}$.

Note: We chose tolerance level 10^{-8} to set a reasonably strict threshold that balances numerical stability and convergence efficiency

Result: The power method converged in 58 iterations for fixed G. It gave the same ranking as the eigenvector method.

4.4 Adding Teleportation

We used $\alpha = 0.85$ and constructed:

$$G_{\sim} = \alpha G + (1 - \alpha)E$$

Which means:

85% of the time: the user follows a link.

15% of the time: the user randomly jumps to another page.

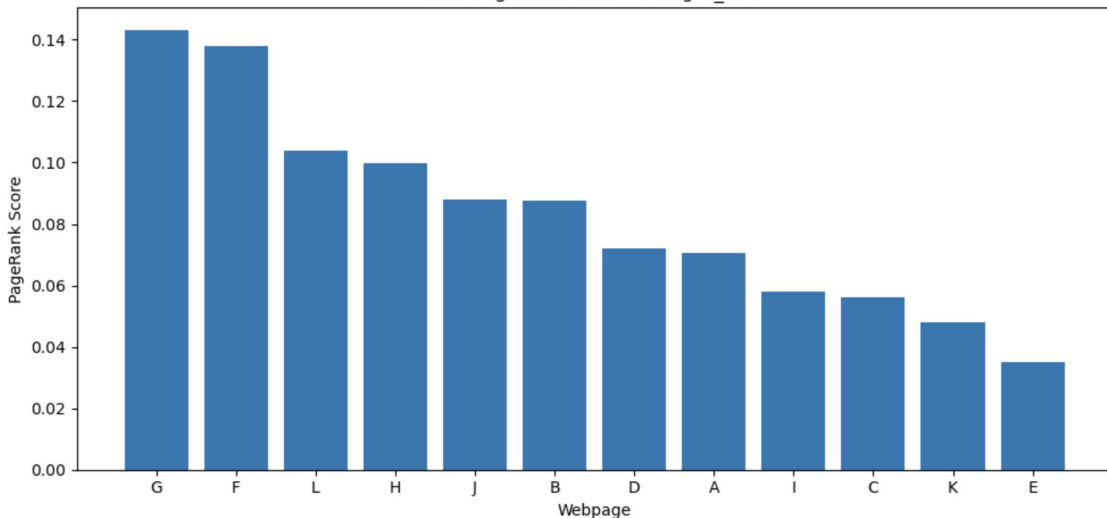
Note: We selected 0.85 for α to reasonably model the behavior of a random user jumping to a new page. A higher value like 0.9 would mean that the user takes longer to teleport (more iterations to converge mathematically). Additionally, 0.85 is very customary in implementations for such reasons.

The power method converged in 38 iterations for $G_{\sim}$.

4.5 Final PageRank Results Using $G\sim$

After adding teleportation, we solve $G\sim^T\pi = \pi$ and normalize the scores so they add to 1.

Final PageRank Scores Using $G\sim$



	Page	Page Title	PageRank Score	Rank
0	G	Lumber Jack Man	0.143222	1
1	F	Good Wood	0.138070	2
2	L	Sappy Planks	0.103697	3
3	H	Timber!	0.099969	4
4	J	The Warped Board	0.087862	5
5	B	The Weeping Willow	0.087768	6
6	D	Shady Place	0.071915	7
7	A	Trees For All	0.070411	8
8	I	Fine Grains	0.057843	9
9	C	Leaf Fall	0.056025	10
10	K	Evergreen Plants	0.048104	11
11	E	The Prickly Pine	0.035113	12

Top pages: G, F, and L. They are the most important web pages in this network.

4.6 Comparing Fixed G and G~

	Page	Page Title	Score from fixed G	Score from G_tilde	Rank from fixed G	Rank from G_tilde
0	G	Lumber Jack Man	0.152383	0.143222	1	1
1	F	Good Wood	0.142588	0.138070	2	2
2	L	Sappy Planks	0.114481	0.103697	3	3
3	H	Timber!	0.107118	0.099969	4	4
4	J	The Warped Board	0.088366	0.087862	5	5
5	B	The Weeping Willow	0.085920	0.087768	6	6
6	D	Shady Place	0.070686	0.071915	7	7
7	A	Trees For All	0.062054	0.070411	8	8
8	I	Fine Grains	0.054893	0.057843	9	9
9	C	Leaf Fall	0.050324	0.056025	10	10
10	K	Evergreen Plants	0.045524	0.048104	11	11
11	E	The Prickly Pine	0.025662	0.035113	12	12

Observation: The ranking from fixed G and G~ was the same.

Teleportation changed the PageRank scores slightly, but it did not change the final ranking order.

4.7 Query Setup

General Idea:

- We define a **query vector q** , which has length of the unique terms and generally represent the **terms we want with 1**, and **others with 0**. Additionally, we can insert values instead of 0 based on specified logic (e.g. -1 to exclude terms)
- We define a **search result vector d** , which has length of # of pages, where $d = T^T @ q$. In other words, we get the **summed values based on all relevant terms from q** . Our decision to select the matching pages is where the logic for search type comes in, as defined next

4.7 Query Setup

Single: Place 1 in query vector corresponding to term and 0s otherwise; 1s in vector d represent the responsive web pages.

-> Select the pages mapped to 1s in d

AND: Place 1 in query vector corresponding to both terms and 0s otherwise; 2s in vector d represent the web pages containing both terms.

-> Select the pages mapped to 2s in d

OR: Setup is similar to AND, except the selection logic changes. We now focus on the 1s and 2s in the vector d , which represent the web pages containing at least one of the terms.

-> Select pages mapped to 1s and 2s in d

BUT NOT: Place 1 in query vector q corresponding to the term we want, -1 for the term we wish to exclude, and 0s otherwise. In the vector d , 1s represent the pages with only the term we want, 0s represent web pages containing both terms, and -1 represents the pages containing only the term we don't want.

-> Select pages mapped to 1s in d

4.8 Search Query Results

	Query	Final ranked pages
0	Eucalyptus	H, J, A, I, C, E
1	Teak AND Ginkgo	L, A, K
2	Katsura OR Magnolia	F, L, J, B, A, I, C
3	Butternut BUT NOT Cherry	G, L, B

After using the term-document matrix to find which pages matched each query, we sorted only those pages using their PageRank scores.

Result: This gives the final order of search results.

05

Conclusion

Summarizing our Findings

5.1 Conclusion

For this project, we used PageRank to rank 12 webpages based on their link structure.

Key Insight:

Our final ranking showed that Page G (Lumber Jack Man) was the most important page in the network.

The Top 3 Pages Were:

1. G - Lumber Jack Man
2. F - Good Wood
3. L - Sappy Planks

5.2 Main Takeaways

- 1) The raw Google matrix had one dangling webpage, which was Page J.
- 2) After fixing the dangling page, every row of G summed to 1.
- 3) Although adding teleportation with $\alpha = 0.85$, slightly changed our scores, the final ranking order stayed the same.
- 4) Lastly, we used the term-document matrix to rank only the web pages that matched each search query.

Resources

Sources used for Slides

- [1]: Saito, Naoki. "MAT 167: Applied Linear Algebra, Lecture 25", Department of Mathematics, University of California, Davis, 5 June 2017.
- [2]: Schwertl, Max. "What Is PageRank and Why Was It Important? - WP SEO AI." WP SEO AI, 3 July 2024, wpseoai.com/blog/what-is-pagerank-and-why-was-it-important/.
- [3]: "Graph Theory - PageRank Algorithm." *Tutorialspoint.com*, 2026, www.tutorialspoint.com/graph_theory/graph_theory_pagerank_algorithm.htm.
- [4]: "PageRank Explained: Theory, Algorithm, and Some Experiments | Hippocampus's Garden." Hippocampus's Garden, 2020, hippocampus-garden.com/pagerank/#power-method. Accessed 5 June 2026.
- [5]: Desmos. "Desmos | Matrix Calculator." *Www.desmos.com*, www.desmos.com/matrix.
- [6]: "Create Graph Online and Find Shortest Path or Use Other Algorithm." Graphonline.top, 2019, graphonline.top/.
- [7]: Langville, Amy, and Carl Meyer. "Deeper inside PageRank." *Internet Mathematics*, vol. 1, no. 3, 1 Jan. 2004, pp. 335–380, <https://doi.org/10.1080/15427951.2004.10129091>.
- [8]: Varn, Dowman. TermDoc03.
- [9]: Varn, Dowman. GoogleNet10.

Credits for Slides: "Differential Equations - Maths - 12th Grade." *Slidesgo.com*, 2026, slidesgo.com/theme/differential-equations-maths-12th-grade#search-math&position-22&results-648&rs=search. Accessed 4 June 2026.

Sources used for Code

- Saito, Naoki. "MAT 167: Applied Linear Algebra, Lecture 02", Department of Mathematics, University of California, Davis, 2017, Accessed 30 May 2026.
- Saito, Naoki. "MAT 167: Applied Linear Algebra, Lecture 25", Department of Mathematics, University of California, Davis, 2017, Accessed 31 May 2026.
- "Return the eigenvector corresponding to the max eigenvalue of A", Stack Overflow, Accessed 30 May 2026, <https://stackoverflow.com/questions/53307152/return-the-eigenvector-corresponding-to-the-max-eigenvalue-of-a>.
- Kumar, Akash. "Power Method - Determine Largest Eigenvalue and Eigenvector in Python", GeeksforGeeks, 30 April 2026, Accessed 30 May 2026, <https://www.geeksforgeeks.org/python/power-method-determine-largest-eigenvalue-and-eigenvector-in-python/>

Thanks!

Do you have any questions?